2026

Educational Network Simulator

TUS: MIDLANDS
MARKUSS ZAKSS

Table of Contents

Table of Figures

INTRODUCTION

Background

Simulation Tools

Networking tools serve a huge purpose in the tech industry, as they allow users to test out networks or device functionality before implementing them into the real world. They allow the admins to visualize the network and test it which can save time as they'll have an idea of how the network will be structured, as well as money because it saves investing in equipment that they might have too much of or not enough of if not estimated correctly. Different tools include Cisco Packet Tracer and OMNeT++ that are both used to simulate networks but have different purposes and functionality.

Cisco Packet Tracer

Cisco Packet Tracer is a network simulation tool focused on mostly training and certifications such as the CCNA and CCNP certificate. The tool strictly focuses on the functionality between Cisco devices with strict protocol logic as you use prebuilt devices to create a network. It also simulates CLI behaviour for the configuration of devices. The main purpose of this tool is as an educational tool for college students and to train people on the basics of networking, making it easier to use than other tools, but limits the scale of the tool to common enterprise networking.

OMNeT++

OMNeT++ is a more advanced networking tool that focuses industrial research into newer protocols and systems. The tool is vendor neutral, meaning it models any hardware, protocol, or system to allow for flexibility and diversity. Being built on C++ it allows the user to modify their own protocols, further complimented by the high precision for timing and traffic analysis provided by the tool. OMNeT is designed for researchers to invent and test newer protocols as apposed to teaching, meaning this tool is a lot more complex with a steeper learning curve requiring the user to have good knowledge in C++ and NED language to build custom modules. OMNeT can scale to massive complex systems, through INET, Simu5G and Veins, which is why the tool is popular among researchers and network engineers.

Problem

Complexity

The main problem with these tools is that they are complex, and even a tool like packet tracer can still be a little confusing to newer people getting into networking. Although there are tutorials and guide sheets on how to work with packet tracer, some of those you must find and other courses that might be useful to people to learn are hidden behind a pay wall. Some people also may not understand why parts of their network are not working, which although is accurate to the real world and forces troubleshooting skills to be developed, it can lead to frustration to newer people who first want to learn the basics before dealing with these common problems.

Exposure

The other problem of these tools is that there isn't any networking tool used or provided for secondary schools, which leads to students not knowing of these types of fields that exist. If a tool like this existed as a short module in secondary schools, students could then discover if this area of IT is an area they would like to learn more about or peruse, potentially leading them to become more interested and study a course related to networking in college.

Aim

The aim of this project is to create an educational networking tool that is easy to use while providing most of the important networking functions needed to configure a network. It will function similarly to Packet Tracer, while being even more restricted to allow for more simplicity and not to overwhelm the user with options.

Objectives

- Create a networking simulation tool like Packet Tracer in functionality
- Restrict functionality to allow for more simplicity
- Provide a visual demonstration of the network created
- Allow the configuration to be saved for later use
- Provide feedback to the user when network connectivity fails
- Develop a web page for the simulation tool

Scope

Included Features

The tool has certain things it should be able to similarly to other networking tools. The tool will provide the user with the options to create a network and simulate connectivity between the devices, as well as different configurations processes that must be followed to allow for proper connectivity. The tool will allow for the configurations of VLANs, EtherChannel, HSRP, and ACLs. The tool will also allow the user the visualize the network to see how it would look like, and to save the configuration to be accessed later.

Limitations

The tool will have certain restrictions in place and limitations on what can be done. You can't configure IPv6 addresses on the ports. Pinging is restricted from VLAN addresses as the tool was made based on pinging ports. Functions such as NAT translation or DNS aren't supported. Can't assign certain IP addresses such as 0.0.0.0 and 255.255.255.255 as they are for specific functions that have not been configured to work in this tool. Ping is the only protocol that can be used to test connectivity.

Overview

The project created in Python, will allow the user to add devices and configure them such that they can be turned on, have IP addresses, and connect to other devices. Those devices can then ping each other to test connectivity, and if failed, it will tell the user why they failed to connect. The use will then have the option to visualize the network so they can see how it might look like. They can also save the network to a database where all the configurations will be saved and later can be loaded back in.

LITERATURE REVIEW

Source 1

Gelenbe E, Nakip M. IoT Network Cybersecurity Assessment with Associated Random Neural Network. *IEEE Access*. 2023; 11:85501–85512.

Summary

This paper proposed a method to assess security in IoT networks by discovering compromised devices and IP addresses using an Artificial Neural Network approach. The authors used an ARNN architecture composed of two interconnected sub networks to represent opposing states where one was compromised, and one wasn't. The model helped evaluate the devices in a network simultaneously. The ARNN is trained to learn relationship and interdependencies between the network nodes and can operate in an online mode using traffic metrics for incremental learning. The results were evaluated by using attack data and comparing it to existing machine learning and detection techniques. The results have shown that ARNN outperforms previous approaches by a significant margin. The study has outlined the effectiveness of neural network-based models and how useful they can be for improving the security of IoT networks.

Integration

The source is relevant to my project as it also focuses on building an interactive environment to simulate a network, though it prioritises simulating for security analysis while the project is focused on being an educational tool. The paper highlights how IoT networks consist of multiple nodes that are analysed through behaviour patterns, while my project consists of devices and connections, where the devices themselves act as nodes especially when visualised with Matplotlib. The paper also supports the importance of understanding how changes within a network can affect the overall system behaviour. The main difference is the paper focuses on a neural network-based approach to detect compromised devices; my project focuses on allowing uses to create their own environment and visualise the networks for learning. This paper helps show the importance of network structures beyond just simulations.

Source 2

Bhola A, Jain A, Lakshmi BD, Lakshmi TM, Hari CD. A Wide Area Network Design and Architecture using Cisco Packet Tracer. *2022 International Conference on Engineering and Emerging Technologies (ICEET)* . 2022.

Summary

This paper goes over the design and implementation of WAN using Cisco Packet Tracer to simulate real world connectivity. The authors explain how WANs are essential in the modern era as they enable communication and data transfer between distributed devices over a geographical area by connecting multiple LANs together. As the research has shown, WANs are important across education, healthcare, and general internet-based communication. The proposed implementation has designed a network topology to represent connectivity in the real world by allowing devices to communicate between LANs and WANs. Application management and email communication was demonstrated within the simulated environment. Packet Tracer was then used to evaluate the performance by monitoring the efficiency of data transfer and the reliability of communication between the devices. According to the results, simulated WAN supports internetwork communication and has provided a practical model for understand networks at a larger scale.

Integration

This source is a relevant to my project as it demonstrated the use of Cisco Packet Tracer to replicate real world internet communication between devices, which is what my project heavily focuses on but for educational purposes specifically. Both systems share a common goal of modelling the behaviour of devices in a controlled environment. Features such as data transfer and email communication simulated in WAN environments relate to some of the networking functions configured in my own project such as connection management and device configuration. The key difference between my project and the study is the study is more focused on WAN infrastructure with packet tracer while my project is a more general-purpose tool for learning and interacting with an environment where users can create and modify devices before visualizing how the network would look like.

Source 3

Komosny D, Rehman SU. Survival Analysis and Prediction Model of IP Address Assignment. *IEEE Access*. 2020; 8:162507–162515.

Summary

The following study investigates the assignment duration of IP addresses with the help of survival analysis which is a statistical method commonly used in healthcare research. The authors have analysed six years of global IP address assignment records to construct a model that can predict how long an IP address may remain assigned to a specific host. Their model has shown improved performance in accuracy of short-term and long-term predictions compared to alternative approaches. The study has also shown that assignment behaviour can vary across ISPs and ASs. The findings can have practical applications to networking domains such as topology mapping, geolocation, and cybersecurity. The model is demonstrated in a fraud prevention scenario where predicted IP assignment duration was used to trigger a two-factor authentication. Datasets of IP address was released by the authors to support further research in this field.

Integration

This source is relevant to my project as it explores the behaviour of IP address assignment over time. Although my project focuses on simulating a network environment rather than a large-scale data analysis, both deal with networking concepts such as IP addresses and network configuration. The paper provides a data driven perspective on how IP address can change across networks, while my project is more about letting the user configure their own network to observe how it functions in a controlled environment. The contrast can be useful as it helps to highlight the difference between theoretical modelling and practical learning tools. The paper also discusses the importance of understand IP addressing systems, which is a core component of my network simulator and how it works.

Source 4

Kurniawan DE, Iqbal M, Adhitya A. Implementation and Analysis of The EtherChannel Technology Using PAgP and LACP Protocols on Cisco Switch Devices. *2021 4th International Conference of Computer and Informatics Engineering (IC2IE)*. 2021.

Summary

The following study investigates the how EtherChannels can improve performance in a network by increase bandwidth and providing reliability within a client-server network. The research addresses issues in network design that can lead to reduced performance. The authors have implemented two link protocols with the EtherChannels, PAgP and LACP. Networks metrics such as delay, jitter, packet loss and throughput were monitored across services such as FTP transfer and video streaming. The results have shown that PAgP performs better in FTP transfer while LACP demonstrated better performance when it came to video streaming scenarios. These findings have shown the difference that the EtherChannel protocol can have depending on the network traffic and application it is used for. This shows the importance of choosing the correct link aggregation protocol to optimise network performance based on the use case of the network.

Integration

This study relates to my project as one of the key functions within it is allowing the user to configure EtherChannel connections. Although not yet configured, protocol choice is important as the study has shown depending on the use case of the network. This paper has also demonstrated the importance of experimenting with designing and configuring a network to discover varying results, which is the main purpose of my project as an educational network simulator in a controlled environment. This source supports the theoretical foundation behind my project by highlighting the significance of network configuration decisions.

Source 5

Yoo HS, Yu WES. Building a QoS Testing Framework for Simulating Real-World Network Topologies in a Software-defined Networking Environment. *2022 5th International Conference on Contemporary Computing and Informatics (IC3I)*. 2022.

Summary

This paper explores the use of a software-defined networking (SDN) simulation framework to evaluate the QoS performance across different network topologies. The authors chose to build upon a previously used model that simulated a fat-tree topology by extending the support to additional network strictures like mesh networks and datasets from Internet Topology Zoo. The study has compared round trip times and HTTP traffic transfer rates through the user of a class-based queuing system. The results have shown that round trip times remain mostly consistent, but data transfer rates decreased drastically the more complex the topologies got. The likes of Uni-C topology required so much more time for network flow instillation that it couldn't complete the benchmarking tests due to limitations. The findings of this research have shown the effect that network complexity can have on network behaviour, The study demonstrates that realistic topology simulation is important for accuracy when it comes to modelling network performance in an SDN environment.

Integration

While my project focuses on building an interactive education simulator for networking, the study focusing on examining the performance of different network topologies in a software-based simulation framework means they both share a goal of modelling the behaviour of a network in a controlled environment. The paper highlighted how topology complexity can significantly affect metrics like data transfer rates, which is like my project where users can create network topologies and observe how different design choices can influence the networks behaviour. The findings reinforce the importance of accurate topology representation, which my project focuses on through visualising the network through NetworkX and Matplotlib. The difference between the source and the project is the source's focus on performance benchmarking in an SDN environment compared to my project that focuses on an education interaction rather than a performance evaluation.

REQUIREMENTS ANALYSIS

Functional Requirements

- The tool must be able to create and delete devices as per user inputs. The function must be made in such a way that the devices are treated as objects, and when removing them the function should delete the object, not just remove it from the list of objects on the network.
- The tool must be able to assign IP addresses to the device ports, EtherChannel, and VLANs as per user inputs. The IP address and subnet mask must also be validated to ensure they are within the correct ranges between 0 and 255 per octet, and the subnet masks, if converted to binary, bust consist of consecutive 1s followed by consecutive 0s. The default gateway must also be on the same subnet as the IP address of the device it is assigned to.
- The connection between the devices must follow a set of rules before allowing them to connect. The devices trying to connect must be turned on and physically connected. The devices must also have a valid IP address on them or have an intermediate device that has correctly assigned IP addresses to allow for a connection to devices on separate networks. If VLANs are configured, then the ports on end devices must be on the correct VLANs to allow for connectivity between devices.
- The tool must allow the user to save the network devices and configurations to a database; with whatever name the user provides. The user must then be able to later load the network based on the selected name, with the network and all the device configurations being correctly loaded back into the tool.

Non-Functional Requirements

- **Usability:** The user should have plenty of networking options when simulating their network while restricting them from being overwhelmed. The type of devices is limited to limit creation to simple networks. The tool will be a simple menu with obvious self-descriptive options for the user to choose from, so they know exactly what they're doing.
- **Performance:** The tool should be fast and responsive, ensuring there isn't a high time delay between the inputs provided by the user. The benefit of python is that it's an efficient programming language, so the outputs come

quickly, allowing the user to create a network and test it quickly. The main issue with performance is the visualizer, which struggles to map the network cleanly the more devices that are added, leading to overlap and messy cross connections.

- **Scalability:** The tool allows for good scalability as you can constantly add devices to the network and configure them as see fit, however with some limitations. The show function configured shows the configurations of all the devices, making it difficult to find the devices you are looking for when the network becomes bigger.

System Constraints

- **Technology Constraints:** A constraint of the system is python, as by itself it can't create a physically interactive application without the use of other external programmes such as FastAPI and HTML.
- **Visualisation Constraints:** Within Python another constraint is the NetworkX and Matplotlib tools for the visualizer, which as previously mentioned can become messy when visualizing a network with many devices on it.
- **Database Constrains:** Another constraint is using MySQL which is a more complex and structured database tool but can be restrictive. This makes it more difficult to add more properties to devices, as entire tables must be dropped and remade with new columns. It can be argued that the use of a NoSQL tool such as MongoDB or Neo4j would be more suitable options for storing the data of the network.

SYSTEM ARCHITECTURE

The system architecture is designed in such a way that as to allow different technologies to be responsible for different components of the system. The system itself consists of Python as the backend, MySQL as the database storage, visualization tools to display the network, and a web interface built on HTML using FastAPI.

Python

Python serves as the backend of the tool, where all the functionality is configured to allow the user to use the tool. The project was made within a number of files, all with their dedicated purposes such as the main menu that the user accesses, a file for the IP address logic, a file for the functions used in the menu, a file for the creation of the device objects, a file for the database logic, and a file for the visualization logic. Python provides the user with the main functions for creating the devices, configuring IP addresses on them, connecting them physically to each other

MySQL

MySQL serves as the database where all the device objects and device configurations are stored. Upon connecting to a system with MySQL on it, the user can input a name for the database which will create one or use it if it already exists. The user can then display the tables created in the database before they load the network into python with all the objects and configurations. The user can also store the current devices and configurations into the database, which will override the existing tables.

NetworkX & Matplotlib

NetworkX and Matplotlib are library tools within Python that are responsible for the visualization of the network in the tool. It allows the user to visualize all the devices currently loaded on the network and see connections between them, including port connections between the devices and IP addresses on the devices. The ports will also display whether they are EtherChannel or regular port connections. It provides a hierarchy for the devices in the visualizer, as the level the devices are displayed on goes as follows; end devices, layer 2 switches, layer 3 switches, firewalls, routers.

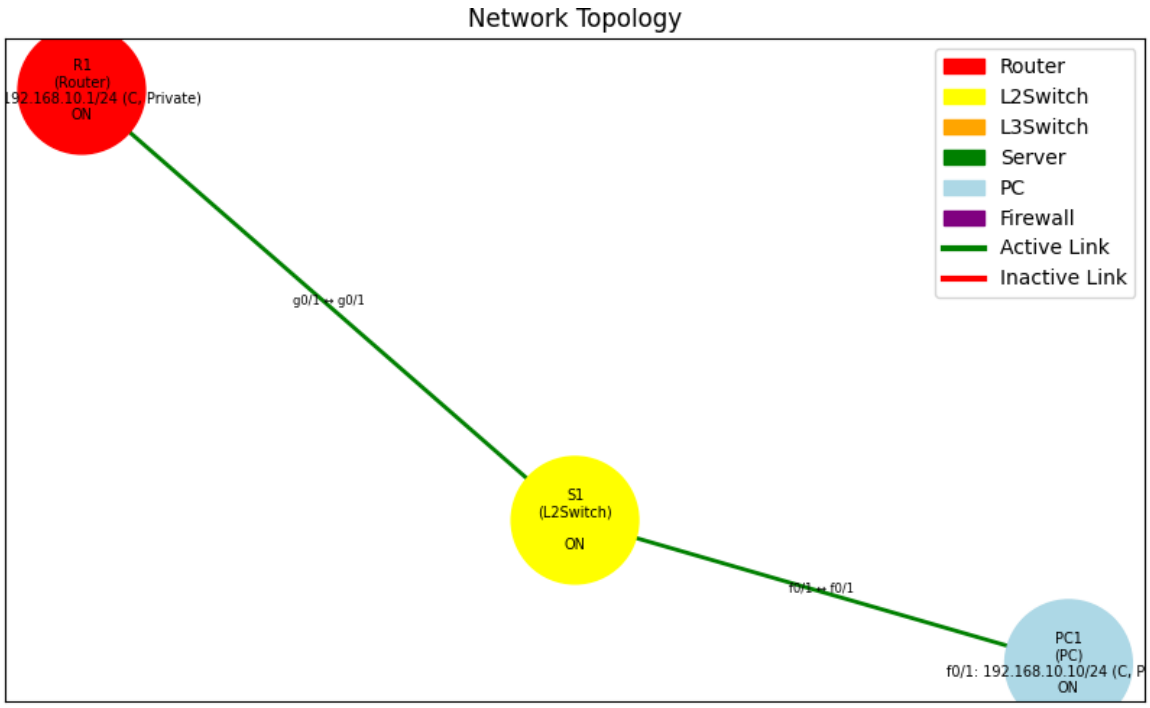


Figure 1- Visualizer Hierarchy

FastAPI & HTML

The purpose of FastAPI and HTML is to provide a web interface for the user instead of using a Python based CLI to simulate the network. FastAPI provides a basic web view of the menu to test out the functionality and ensure they are all working as intended before applying those functions into HTML where they can be used to create a visual and interactive GUI for the user to use.

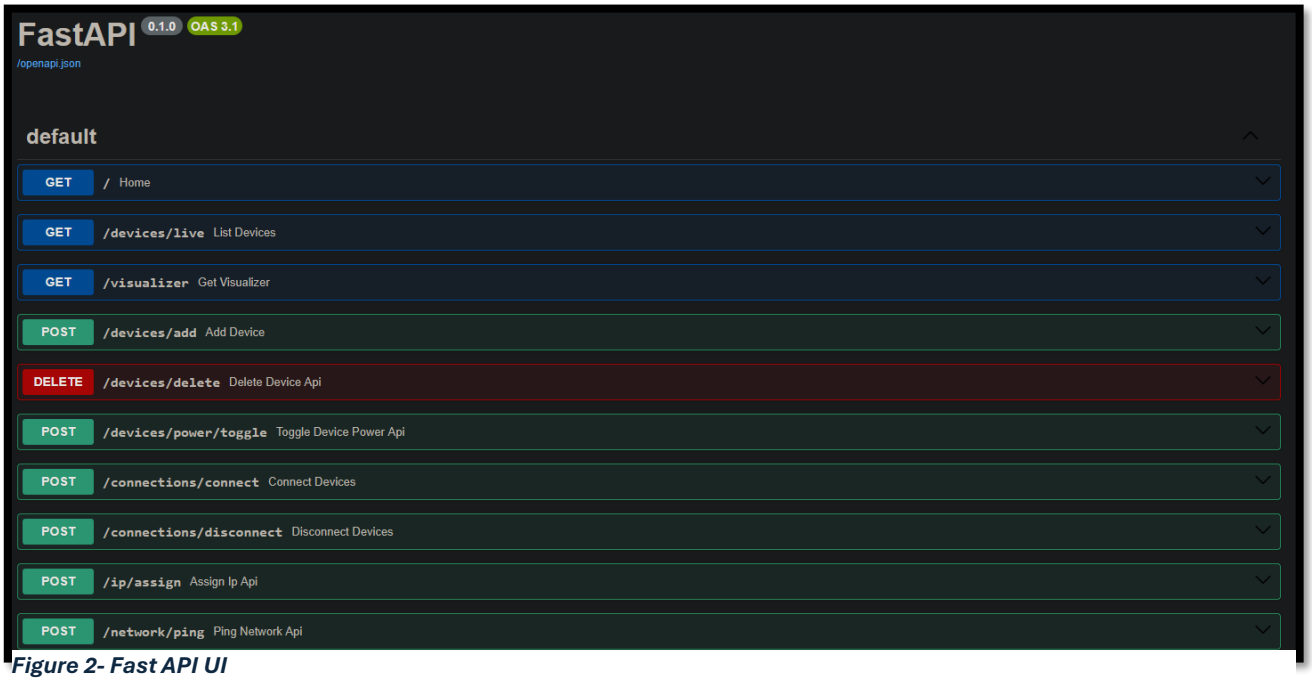


Figure 2- Fast API UI

Flow Chart

The system flow chart demonstrates the way the user interacts with the components and interface. The use inputs are processed by the Python backend before being executed as a function or to store the configurations to a database. It also demonstrates how the visualizer is generated and presented to the user.

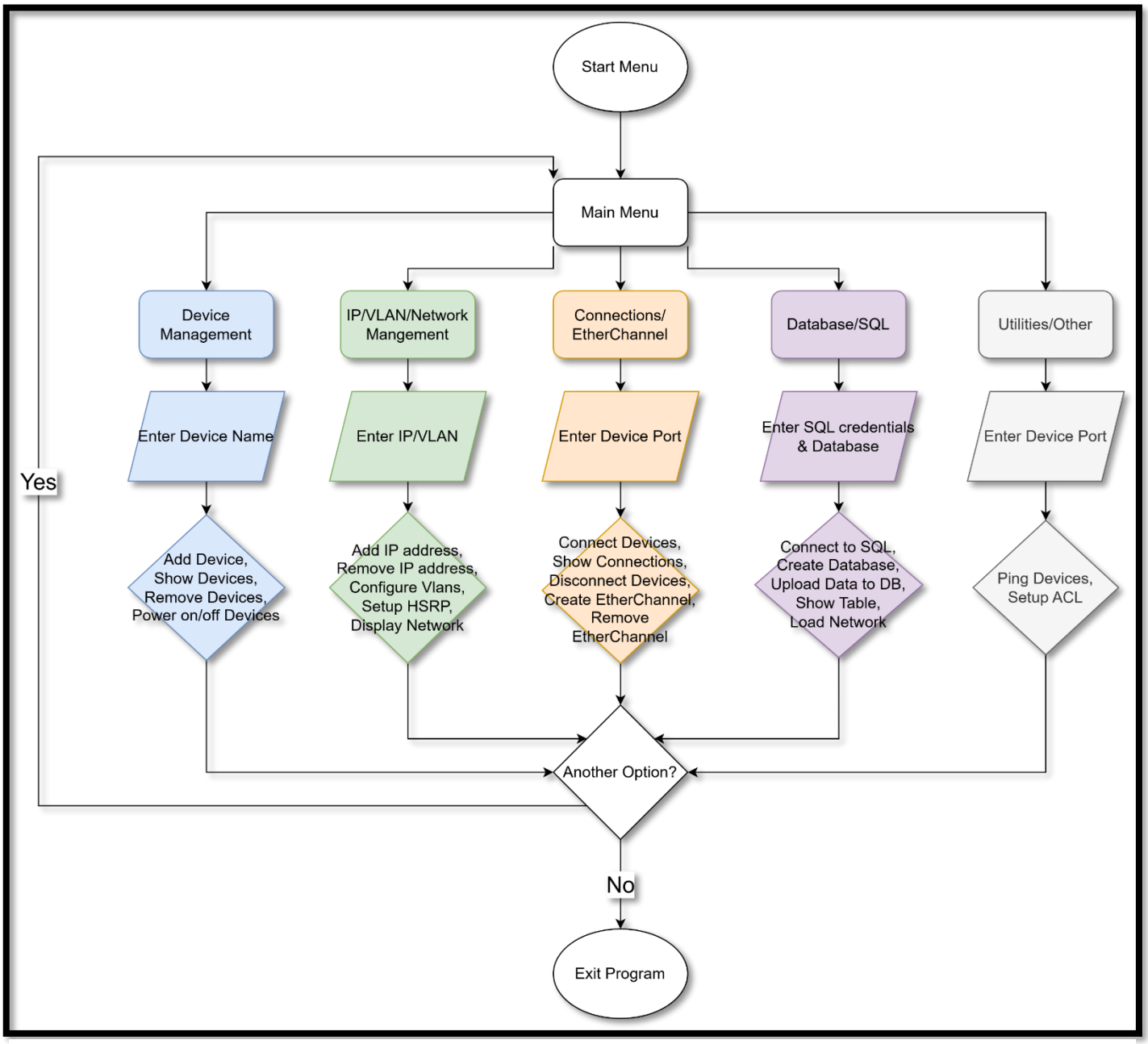


Figure 3- Flowchart

IMPLEMENTATION

Development Approach

Initial Development

Progress during the initial stages of the project consisted of brainstorming ideas on what the project should be, which at first it was an IP address calculator. The following step was to construct a Gantt chart to provide an idea as to the schedule that would be attempted to follow throughout the year through the construction of the project. When the project began, it was decided that Python would be used to create the backend of the project.

The following stage was coding the IP address logic, which consisted of checking to ensure the IP address and subnet mask were correct when inputted, and converting them into binary for other use cases. The beginning of this project was very slow as ideas for the project were still in development, as the only idea at this stage was an IP address calculator accessed by a web page.

```
Select option: 1

Devices on the network:
PC1
S1
L1
Select a device to assign an IP to: PC1
Assign IP to a (P)ort or (E)therChannel pr (V)lan? [P/E/V]: p

Available ports:
f0/1
Select port to assign IP to: f0/1
Enter IP address: 192.168.10.20
IP set to: 192.168.10.20
Enter subnet mask or prefix (e.g., /24 or 255.255.255.0): /24
Network Address: 192.168.10.0
Broadcast Address: 192.168.10.255
Valid Host Range: 192.168.10.1 - 192.168.10.254
Wildcard Mask: 0.0.0.255
Suggested Default Gateway: 192.168.10.1
Enter default gateway (press Enter for suggested):
Default Gateway set to 192.168.10.1
```

Figure 4- IP Address Assignment

Core Feature Development

As the project progressed it was soon realized that it was turning into a network simulator like Cisco Packet Tracer. Following the creation of an IP address checker file, a devices file was made, responsible for the creation of devices on the network, and assigning properties to them through an OOP (Object-Oriented-Programming)

approached. Additionally, factory patterns were used for simpler repetitive creation of objects by forcing their creation to only occur when asked for by the user.

```
Select option: 1
What device would you like to add (PC, Server, Router, L2Switch, L3Switch, Firewall): L2
What is the device name: S1
Choose a model ['2960', '2960S']: 2960S
L2Switch 'S1' (model 2960S) added to the network
```

Figure 5- Device Creation

A function file then followed that would be responsible for the various functions for the user to be able to execute, which at first consisted of simple assignment of IP addresses, changing device state between on and off, and connecting devices physically to each other. The biggest function to be added is the ping function, as this required continuous updating as newer networking features were added to the tool.

```
Select option: 1

Devices: PC1, PC2, PC3, PC4, S1, S2, L1, L2, R1, F1, S3, Web Server
What device do you want to ping from: PC1

Available ports with IPs:
f0/1 (192.168.10.10)
Which port or EtherChannel are you pinging ping from: f0/1
What device do you want to ping to: Web Server

Available ports with IPs:
g0/1 (172.16.40.10)
Which port or EtherChannel are you pinging ping to: g0/1

Checking route...

Route taken:
PC1 → S1 → L2 → R1 → F1 → F1(172.16.40.1)

Pinging 172.16.40.10 with 32 bytes of data:
Reply from 172.16.40.10: bytes=32 time=1ms
Reply from 172.16.40.10: bytes=32 time=1ms
Reply from 172.16.40.10: bytes=32 time=1ms
Reply from 172.16.40.10: bytes=32 time=1ms
Ping successful! PC1 can reach Web Server
```

Figure 6- Successful Ping Execution

The next big feature to be added was a visualizer, which used NetworkX and Matplotlib. The implementation of this function was to take the devices existing on the network and display them visually to show the connections between them and details on each device such as IP addresses, device state and device name.

Following this it was decided that a way to store the configurations was a requirement, not only for the user to save their network, but for testing as repeatedly needing to create a network from scratch for testing wasn't an efficient use of time during the creation of the project. For this, MySQL was chosen as a reliable application for storing data on a database.

DEVICES TABLE:

id	name	deviceType	state	totalPorts	usedPorts	unusedPorts	model
1	PC1	PC	on	1	1	0	Null
2	PC2	PC	on	1	1	0	Null
3	S1	L2Switch	on	26	2	24	2960
4	S2	L2Switch	on	26	3	23	2960
5	L1	L3Switch	on	28	2	26	3650
6	L2	L3Switch	on	28	4	24	3650
7	R1	Router	on	3	3	0	2911
8	L3	L3Switch	on	56	4	52	9300
9	S3	L2Switch	on	26	3	23	2960
10	PC3	PC	on	1	1	0	Null

Figure 7- Device Table from SQL Database

Advanced Feature Integration

As the project continued to progress, more advanced features continued to be added to the function file, such as EtherChannel creation, HSRP, ACLs, and VLAN configuration. However, the addition of these features resulted in the pinging function needing to be modified to allow for these features to properly function.

Upon the completion of this, it was realized that to create a web-based GUI for the simulator using HTML, there needed to be framework to allow it, which is where FastAPI had to come in. This unfortunately, resulted in needing to modify the entire existing code to be suitable for FastAPI use. The project ended in the final stages of FastAPI development before it got to be used in the making of a HTML web-based GUI.

Gantt Chart

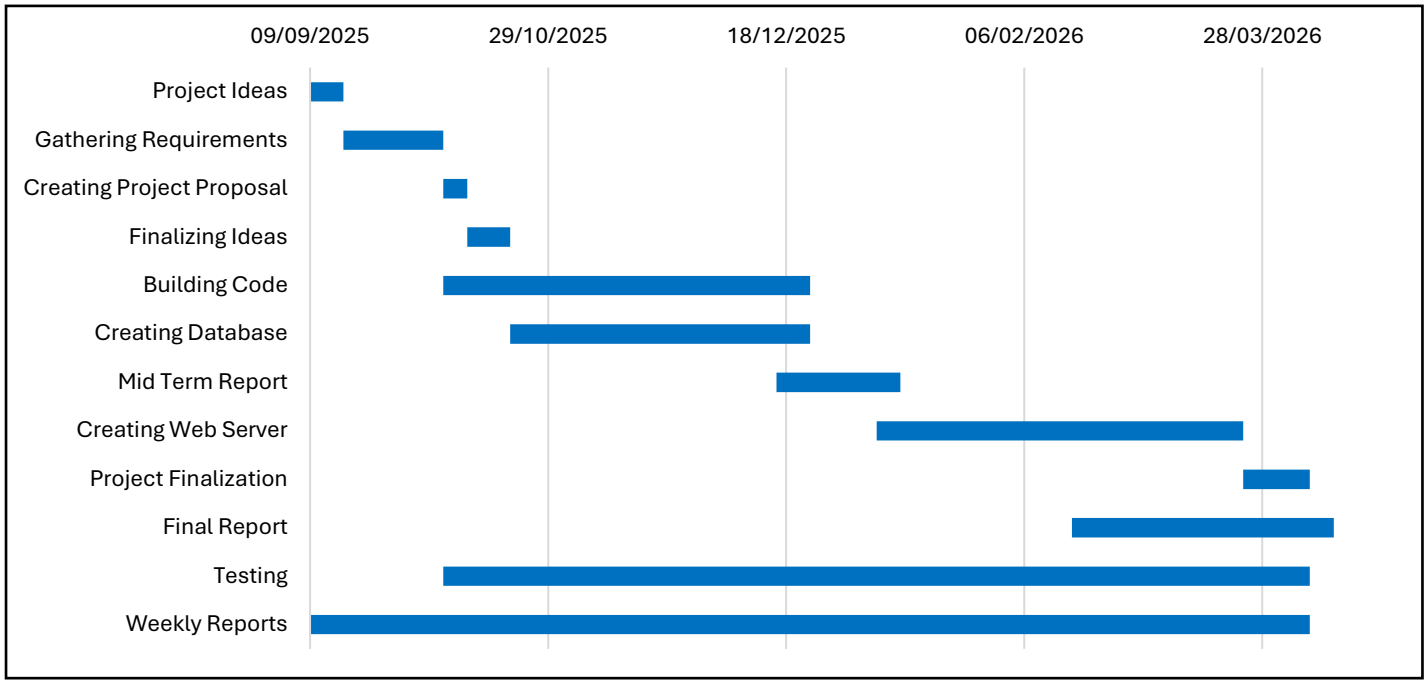


Figure 8- Gantt Chart

Key Features

A key feature in this program is the option to create devices. It was decided that the devices should be treated as objects, with each device having their own similar and unique properties, such as all devices consisting of an on or off state, but not all devices consisting of IP addresses, as layer 2 switches don't assign IP addresses to their ports.

IP assignment is another crucial component as devices must only be assigned the correct IP address. This means they cannot except the 255 limit per octet and subnet masks are required to consist of consecutive 1s followed by consecutive 0s when assigned an IP address.

The ping function is the most crucial function in the system as the user confirms connectivity between devices using it. The function ensures that the devices meet a certain set of criteria before a connection can be established between devices for communication.

The visualizer is another crucial component that allows the user to visualize the network topology after creating it. This is important as an educational tool as without

it, the user won't know what exactly they're making without a visual representation of the network.

The database is an important component that allows the user to store configurations of components on a database. This is important to allow users to return to previously made topologies to either update them or simulate them again. It also saves time during testing as it allows for changes in the code to be made and tested on already existing topologies.

Code Structure

The code structure consists of seven files. The main code file presents the user with options on different sets of tasks they might want to do, or to access the visualizer. When selected, another file is accessed that contains a set of options to choose what task the user might want to complete. Most of the options lead the user into another file that contains all the available functions for the user to execute such as IP address assignment, device removal, show functions, pinging, etc. A set of these functions leads to the database file that is used to connect to MySQL on a machine provided the name, port, password and IP address. Once connected successfully, the user may save their topology to the database or load their network from the database. Both the database file and function file import the devices file and IP checker file, as the devices file is responsible for the creation of the devices as objects when user requests so, and the IP checker file is used to verify IP addresses that are assigned.

The RestAPI version of the code is much the same, except it has another file which instead of providing a menu option, it calls all the existing functions that are created and automatically creates a web page based on whether the function was created as a GET, POST, PUT, or DELETE function. Should the HTML page be made, it would contain the code to create the visual GUI on a web page, with a separate section for calling the REST functions to be used in the GUI for the user to interact with.

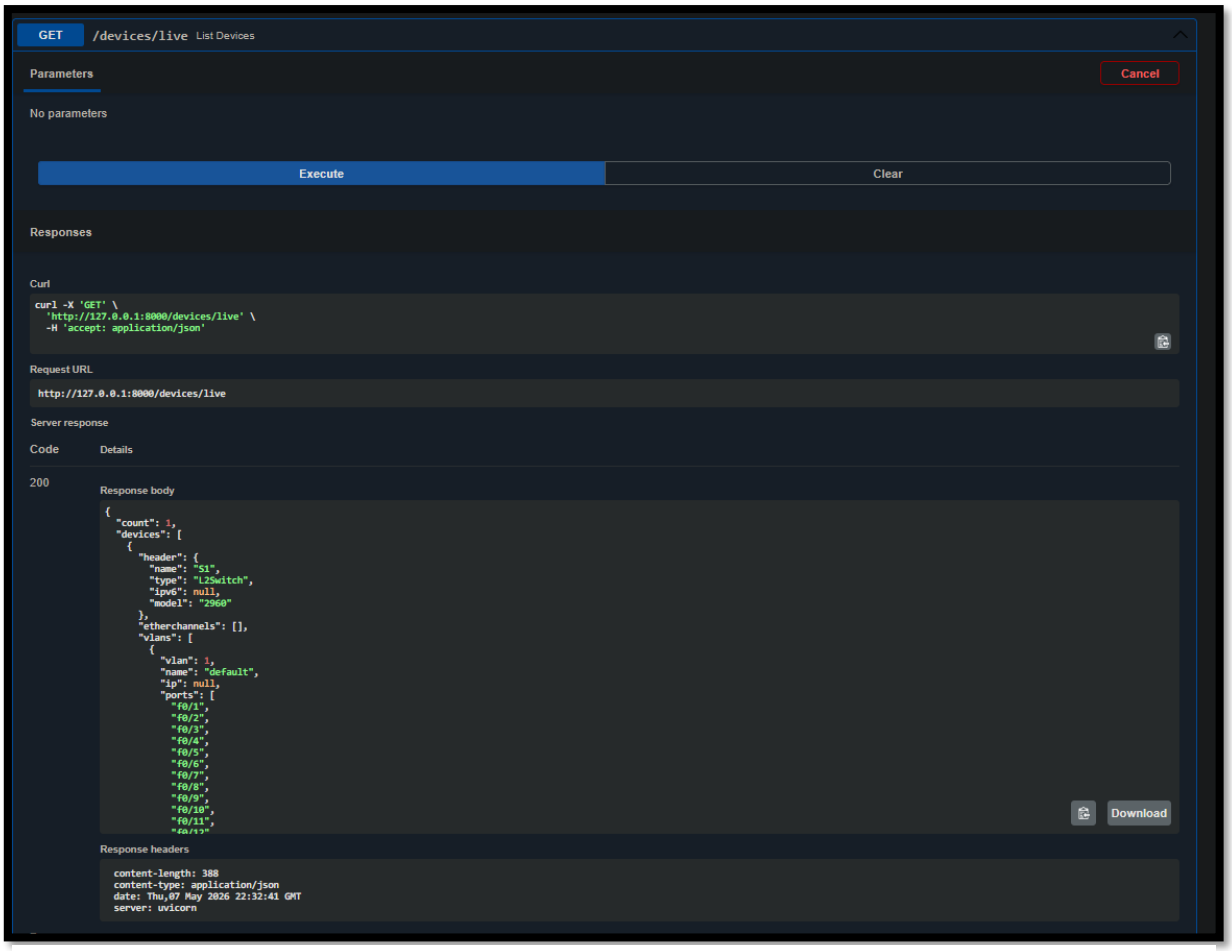


Figure 9- GET Devices Function in FastAPI

Challenges

The main challenge throughout this project consisted of the ping function, as it constantly had to updated to support added features to system. At first it was constructed to identify if devices were connected to each other, but as the project progressed it had to be updated to verify if devices were turned on, if they had an IP address, if said address was correctly configured to allow for a connection, if the IP address was on a port or EtherChannel, if HSRP was configured and if an ACL was configured to block IP connections. These constant updates meant the function had to be retested repeatedly upon each iteration of the project.

Originally the IP assignment function within the system assigned IP addresses to the devices created rather than ports on the devices as the device objects didn't contain ports originally. As the project continued, it was realized that the ports needed to be added to devices, and for realistic pinging connectivity between devices, the IP

address function had to be remade. This resulted in most of the code to be remade to change the logic to support port IP addresses rather than device IP addresses.

The visualizer provided quite the challenge in attempting to provide a clean visual topology of the network. The main issues concerning the visualizer is NetworkX becoming messier and inconsistent the more devices created on the network. Although far from ideal, the visualizer has been adjusted to provide some consistency by forcing a set seed as to ensure each time the topology is visualized, it will remain the same unless device or connection changes are made. Another adjustment was providing a hierarchy, by setting ensuring end devices are at the bottom, with layer 2 switches above them, then layer 3 switches, firewalls, and finally routers. This provided more consistency for the visualizer by allowing the consistency of device layering to remain.

The database construction within MySQL provided a challenge as each additional function added to the system such as HSRP, VLAN and EtherChannel required another table to be constructed automatically within the database code, or existing ones needed to be updated to support these changes.

To allow MySQL to function on the device, XAMPP was used, which resulted in problems whenever XAMPP would block ports used for the database connection. This would result in XAMPP needing to be reinstalled, leading to the database tables to be reset, forcing created topologies used for testing to have to be manually remade.

An unexpected challenge was realizing that to allow RestAPI to be used with the code, the entire code had to be reconstructed to support it separately to the Python CLI. As a result, the HTML web page couldn’t be constructed in time as focus was put to reconstructing the code for RestAPI support.

Time management proved to be a major challenge throughout the project as towards the early stages of the project, minimal progress was made as ideas weren’t as concrete. Once ideas were being realized, the pace picked up, but much later that ideal, resulting in the project not being fully complete with the potential it had.

TESTING AND EVALUATION

Test Plan

The plan for testing the simulator consisted of testing each function added to the simulator as they were added. Any changes made to existing code such as database tables or device object properties also required testing to ensure they remained functional to additional changes. The ping function would remain the main testing function throughout the project as it was responsible for confirming whether device connections were successful. RestAPI testing remained much the same, but testing was done within a web page.

Functional Testing

```
Devices on the network:
PC1
Select a device to assign an IP to: PC1
Assign IP to a (P)ort or (E)therChannel pr (V)lan? [P/E/V]: p

Available ports:
f0/1
Select port to assign IP to: f0/1
Enter IP address: 192.168.1.255
IP set to: 192.168.1.255
Enter subnet mask or prefix (e.g., /24 or 255.255.255.0): /24
IP cannot be network or broadcast address
```

Figure 10- Failed IP Address Assignment

The ping function was tested repeatedly throughout the project through its iterations, where during the initial stages it checked for to see if the device being pinged was connected to it, followed by checking if it is on. As the project progressed the function was updated to check for port IP addresses, intermediate device hops, VLANs, and HSRP. At the final stages the ping function works except for HSRP, where it was only

```
Select option: 1

Devices: PC1, PC2, S1, S2, L1
What device do you want to ping from: PC1

Available ports with IPs:
f0/1 (192.168.10.10)
Which port or EtherChannel are you pinging ping from: f0/1
What device do you want to ping to: PC2

Available ports with IPs:
PC2 has no ports or EtherChannels with IP addresses
```

Figure 11- Failed Ping Execution

in the end realized the function bypasses the need for a default gateway if VLANs are configured.

The storing of the data works as intended, where every configuration is saved to be loaded in later. These tables consist of one for devices, ports, connections, EtherChannel, VLANs, and HSRP. Throughout the project this had to be updated to add newer tables and columns to support the addition of newer functions and properties.

The visualizer was tested originally with just simply visualizing devices as nodes connected to each other, but as the project progressed it was updated to visualize the nodes with details such as name, IP address, and port connections. Throughout the project the visualizer was a common tester to see how the network looked and easily determine if devices were correctly connected. The visualizer component received limited additional functionality to avoid excessive clutter on the nodes, and the limitations with the visualizer such as an inconsistent and often messy layout.

RestAPI testing commenced during the late stages of development, which consisted of adding a GET, POST, PUT, or DELETE function depending on the purpose of the original Python function. Upon each addition of the function, it was tested in the

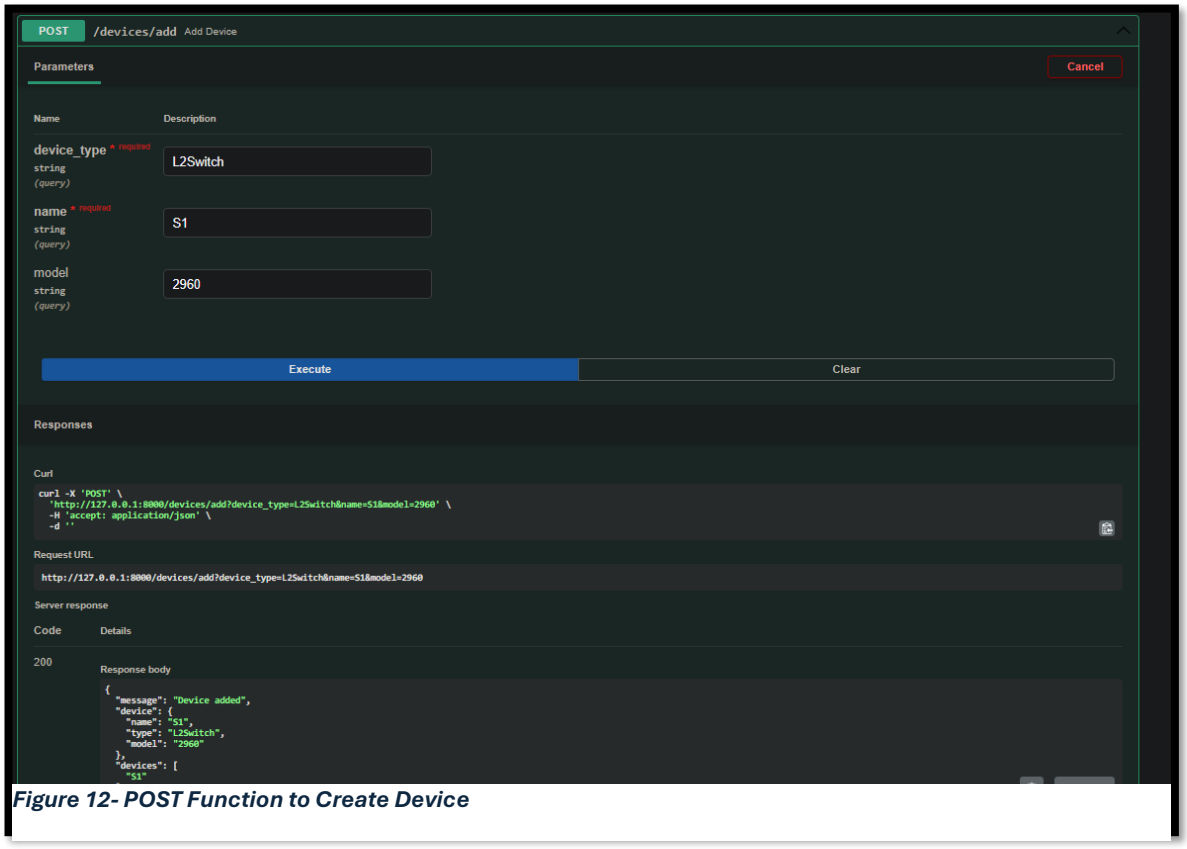


Figure 12- POST Function to Create Device

Swagger UI web page. Most functionality of the RestAPI was successful, except for connection with the MySQL database which still requires further development.

Test Case	Expected Result	Actual Result	Status
Assign valid IP address	IP assigned successfully	Successful	Pass
Assign invalid subnet mask	Error message displayed	Successful	Pass
Ping connected devices	Successful ping response	Successful	Pass
Ping disconnected devices	Error message displayed	Successful	Pass
Save network to database	Network stored correctly	Successful	Pass
Load saved network	Topology restored	Successful	Pass
HSRP ping test	Ping succeeds through virtual gateway	Failed due to logic issue	Partial

Usability Testing

Throughout the development of the project, usability was consistently tested by considering how confusing the user would find the inputs and more importantly, failed inputs. Upon each function testing, responses were added to successful and failed inputs and executions, often informing the user why a function failed to execute. This was most important during the development of the ping function, as majority of users would become confused during connection testing of devices. The ping function provides different error message depending on the error such as for devices not being on, not having an IP address, or not being configured correctly to allow a successful connection even if physically connected.

```
Select option: 1

Devices on the network:
PC1
PC2
What device do you want to ping from: PC1

Available ports with IPs:
f0/1 (192.168.10.10)
Which port or EtherChannel are you pinging ping from: f0/1
What device do you want to ping to: PC2

Available ports with IPs:
f0/1 (172.16.10.10)
Which port or EtherChannel are you pinging ping to: f0/1

Checking network connectivity and route...
Ping failed: no route from PC1 to PC2
```

Figure 14- Failed Ping Execution

Usability could be further improved by implementing loops for certain options, such as switching device state. Upon turning the device on or off, the user is returned to the previous menu but allowing them to turn on multiple devices subsequently can improve usability, even further if allowed to select multiple devices to turn on at once.

```
Select option: 2

Devices on the network:
L1 (L3Switch)
L2 (L3Switch)
Select the first device for EtherChannel: L1
Select the second device for EtherChannel: L2

Ports on L1 connected to L2: ['f0/1', 'f0/2', 'f0/3']
Ports on L2 connected to L1: ['f0/1', 'f0/2', 'f0/3']
Select ports on L1 to include in EtherChannel (comma-separated): f0/1, f0/2, f0/3
Select ports on L2 to include in EtherChannel (comma-separated): f0/1, f0/2, f0/3

EtherChannel 'Po1' created between L1 ports ['f0/1', 'f0/2', 'f0/3'] and L2 ports ['f0/1', 'f0/2', 'f0/3']
```

Figure 13- EtherChannel Configuration

Evaluation

The final state of the project has successfully achieved many of the initial objectives. The project has successfully become a network simulation tool, with functionality like that of Packet Tracer. The tool was successfully restricted to allow for simplicity by limiting options. The project has provided an option for a visual demonstration of the network topology, which although far from perfect, serves its purpose well. Saving and loading of configurations has also successfully been implemented, allowing the user to save their network for later use. Feedback to failed connections is also provided, with descriptions informing the user on the reason for the failure.

The web page development, however, remains incomplete as it remained at the RestAPI testing stage, failing to reach the HTML web page stage, leading to a bare-boned UI without much functionality.

DISCUSSION

What Worked

The project provided many functional features, such as the creation of devices. Each device acts as an object, allowing them to feel more like real devices. It allows for easy modularity as adding properties and functions to the devices becomes easier through creating them as objects and using factory patterns. This is important to ensure that the network feels as realistic as possible.

The ping function provided a useful way for the user to confirm connectivity between devices by testing if two devices on the same or different network can successfully connect to each other. This is a key element of the project that allows it to be a proper networking simulation tool.

The database functionality proved very useful for storing all configurations created on the network to later be used again, allowing the user or tester to save time by not requiring the recreate the same topology repeatedly. This is important to allow for convenience and accessibility of the tool.

The visualizer proved as a useful tool for visualizing the network topology using its hierarchical layout of each device type. This is important as it provides the user with a way to see how the network would look put together.

The tool provides plenty of educational value for newer students or people getting into networking. Its limited functionality and simplicity serve to not overwhelm the user while providing enough freedom to explore the basics of networking. This allows the tool to serve a purpose among other simulation tools of similar purpose.

Limitations

The visualizer, although useful, proved to be cluttered as the network scaled up. This has a negative impact on anyone trying to test a larger network. This limits functionality to smaller and more basic networks.

The HTML GUI remains incomplete, resulting in both a basic Python CLI and RestAPI version of the project. This negatively impacts the project as users may choose applications such as Packet Tracer for its better visual and interactive interface.

HSRP and ACL functionality is still in development, leading to incomplete functionality of the tool as these are key features useful to learning how networking system function.

IPv6 remains absent within the project, leading to limitations within networking as IPv6 is growing in use cases as IPv4 addresses become obsolete as there are too many devices in the world to support all IPv4 address combinations.

Literature Comparison

The completed projected shares many similarities to different studies and networking tools that were discussed throughout the literature review. The main similarities exist within topology simulation and network configuration. However, there are many difference too due to the simplicity of an educational focus and restricted functionality intended for beginners.

The work by Abhishek Bhola et al. on WAN design using Cisco Packet Tracer closely resembles the purpose of this project. Both focus on simulating network communication within a controlled environment to experiment on networking concept without the need for physical hardware. The study provided the use of Packet Tracer to model communication between LANs and WANs. This project has a similar concept but on a much smaller and simpler scale where users can create devices, establish a connection, and ping to test communication between the devices.

The research conducted by Erol Gelenbe and Mert Nakip focused on IoT cybersecurity has different objectives from this project but similarly relies on representing networks as interconnected nodes and analysing relationships between these devices, similarly to NetworkX in this project. The paper reinforces the importance of understanding the structure of a network and device interaction which also forms the key pat of this simulator through the visualizer tool and ping testing function. The main difference of this project from the paper is the paper focuses more on machine learning and security analysis as apposed to an educational simulator with user interaction.

The study by Dan Komosny and Saeed Ur Rehman explored IP address assignment prediction using survival analysis techniques. The paper mainly focuses on statistical modelling and networking datasets but remains relevant to this project due to the emphasis on IP addressing and network configuration. The study highlights the

importance of understand IP addressing behaviour of modern networks, which relates to the importance of IP address assignment and subnet mask validation within this project. The main difference is the paper focuses on analyses of real-world address allocation trends compared to manual configuration in a controlled environment of this project.

The EtherChannel research conducted by Dwi Ely Kurniawan, Mohd Iqbal, and Adhitya Adhitya also closely aligns with functionality implemented within the simulator. The study examined performance between PAgP and LACP protocols with the use of metrics such as delay, jitter, packet loss, and throughput. While the simulator doesn't support traffic analysis or functionality, the study is important for understanding the different purposes of both protocols when configuring EtherChannel connections on a network. The study supports the educational importance of experimenting with network configurations to discover what works best.

The software-defined networking simulation framework developed by Hyeong Seon Yoo and William Emmanuel S. Yu demonstrated how a more complex network can affect the Quality of Service (QoS) performance and simulation behaviour. This directly relates to the challenges faced with the visualizer in the project, where NetworkX struggled to cleanly display a network topology the more complex it became. Similarly, the research paper observed performance reduction within larger network topologies. The main differences stemmed from the paper focusing on QoS benchmarking and performance while the project focuses on educational interaction.

Overall, the revied literatures supported many of the design philosophies of the project, including the topology simulation, IP configuration, EtherChannel support, and educational importance within the simulator. The comparisons also helped highlight the distinction of the project, where rather than focusing on advanced simulations or performance monitoring, it focuses on a simplified education network simulations with a focus on accessibility and intractability.

FUTURE WORK

Near Future

At the projects current stage, future work will initially consist of implementing unfinished functions and completely objectives originally set out to complete. This means the project must first allow for ACL and HSRP functionality to operate as intended. The project will also require for a HTML GUI to be configured as originally stated in objectives.

Upon completion of these tasks, additional features can begin to be implemented such as DNS and NAT support to allow users to simulate connecting to a NAT IP address and a domain name. This would include adding additional connectivity functions aside from ping with ICMP, such as email and web access through SMTP/IMAP/POP3 and HTTP/HTTPS respectively. Another feature to be added would be Remote Access to allow user access to the tool through a web page from outside of the network. This could be achieved through port forwarding and DNS lookup to give the web page a name rather than an IP address. IPv6 address support is another required feature to implement to allow simulations of modern network environments and protocols.

Long-Term

Long term the project has potential to develop from a web page to a downloadable application. The long-term plan consists of creating an interactive downloadable app that allows users to both drag and drop components, as well as use a CLI to create the network. The application would also provide guides for newer people from creating a small basic network to a final guide creating an expanded network using all the features provided within the tool such as VLANs, routing, EtherChannel, and redundancy protocols. Upon completion the user can be prepared to learn Packet Tracer and progress with developing their networking skills.

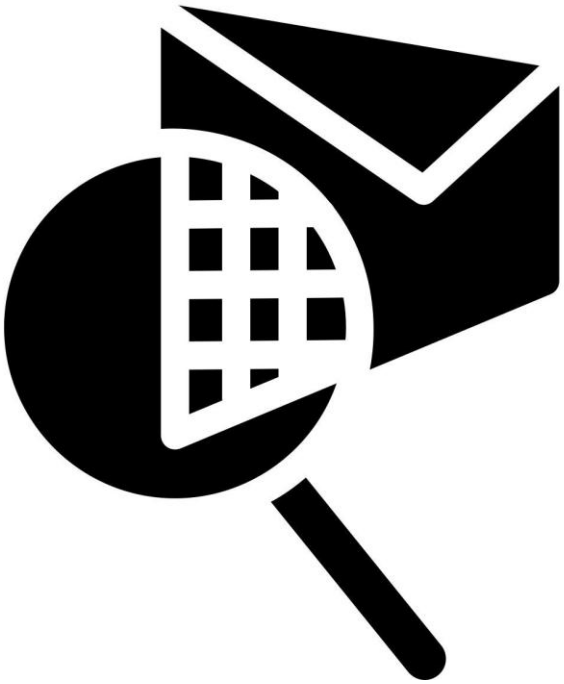
CONCLUSION

In conclusion, while some advanced features remained in development, the project has provided a good template for future development and improvement that may result in the project becoming something useful to people getting introduced into networking. The system allows the user to create and configure devices, assign IP addresses, test network connections, visualize network topologies, and store configuration for later use.

Despite several features remaining under development, the resulting project has provided a simpler alternative to existing simulation tools using a Python CLI, MySQL, NetworkX, Matplotlib, and Fast API that allows for network creation and additional functionality in future iterations.

The project has demonstrated the importance of a useable and accessible network simulator by simplifying functionality through restrictions while ensuring that essential networking concepts remain intact.

The project has turned out a success and provided a future idea for development with potential to become useful to newer students or secondary school students who may have interest in the computer networking field.



REFERENCES

Bhola, A., Jain, A., Lakshmi, B.D., Lakshmi, T.M. and Hari, C.D. (2022) 'A Wide Area Network Design and Architecture using Cisco Packet Tracer', *2022 International Conference on Engineering and Emerging Technologies*.

Gelenbe, E. and Nakip, M. (2023) 'IoT Network Cybersecurity Assessment with Associated Random Neural Network', *IEEE Access*, 11, pp. 85501–85512.

Komosny, D. and Rehman, S.U. (2020) 'Survival Analysis and Prediction Model of IP Address Assignment', *IEEE Access*, 8, pp. 162507–162515.

Kurniawan, D.E., Iqbal, M. and Adhitya, A. (2021) 'Implementation and Analysis of The EtherChannel Technology Using PAgP and LACP Protocols on Cisco Switch Devices', *2021 4th International Conference of Computer and Informatics Engineering*.

Yoo, H.S. and Yu, W.E.S. (2022) 'Building a QoS Testing Framework for Simulating Real-World Network Topologies in a Software-defined Networking Environment', *2022 5th International Conference on Contemporary Computing and Informatics*.

Cisco Networking Academy (n.d.) *Cisco Packet Tracer*. Available at: https://www.netacad.com/cisco-packet-tracer (Accessed: 10 April 2026).

OMNeT++ Community (n.d.) *Introduction to OMNeT++*. Available at: https://omnetpp.org/intro/ (Accessed: 10 April 2026).

Python Software Foundation (n.d.) *Applications for Python*. Available at: https://www.python.org/about/apps/ (Accessed: 21 April 2026).

FastAPI (n.d.) *First Steps - FastAPI Tutorial*. Available at: https://fastapi.tiangolo.com/tutorial/first-steps/#check-the-openapi-json (Accessed: 3 May 2026).

Oracle Corporation (n.d.) *MySQL Storage Requirements*. Available at: https://dev.mysql.com/doc/refman/9.7/en/storage-requirements.html (Accessed: 22 April 2026).

NetworkX Developers (n.d.) *NetworkX Documentation*. Available at: https://networkx.org/en/ (Accessed: 21 April 2026).

APPENDIX

Appendix A – Additional Screenshots

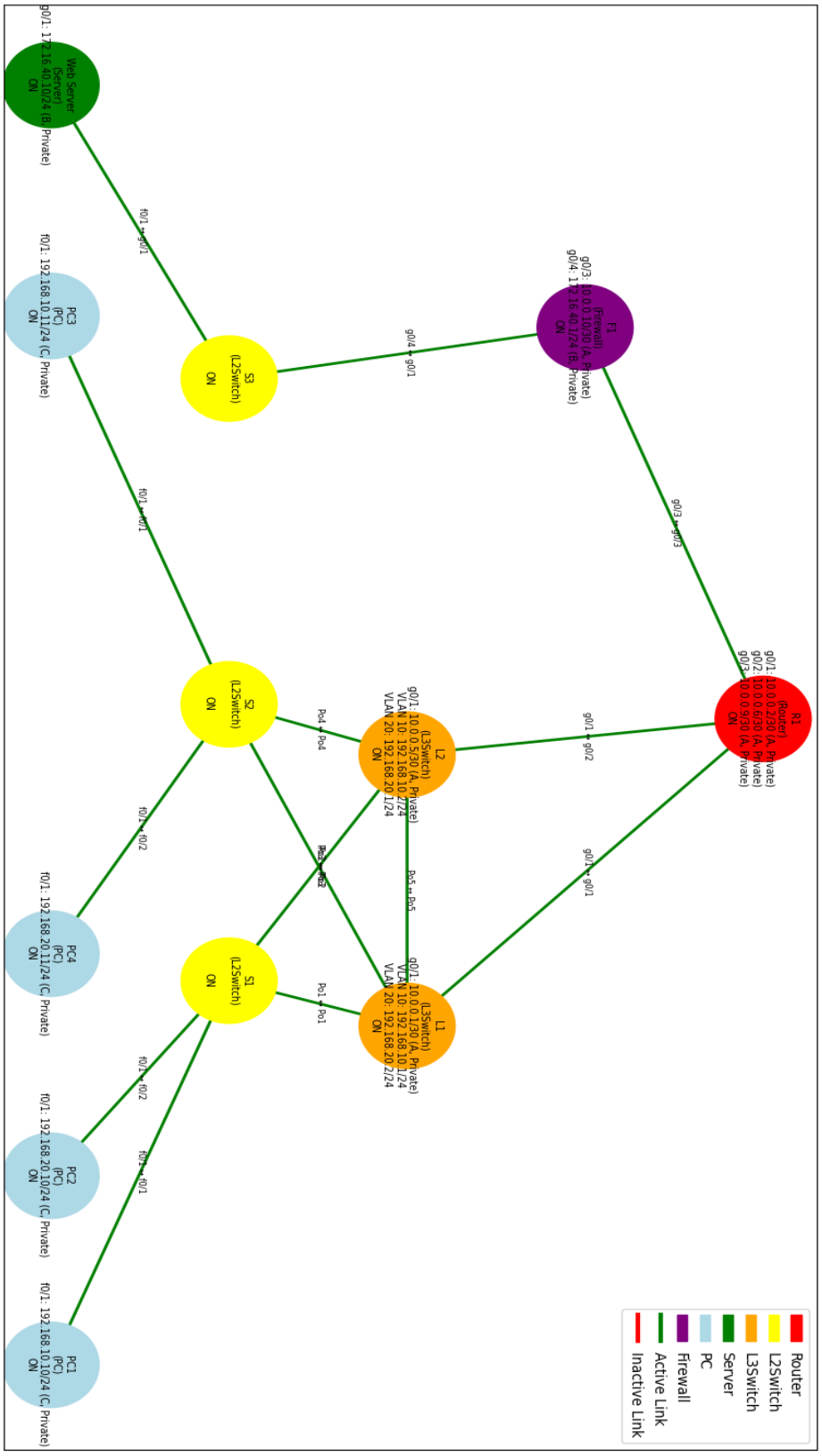


Figure 15- Visualizer of Network Topology

```
Select a device to assign an IP to: PC1
Assign IP to a (P)ort or (E)therChannel pr (V)lan? [P/E/V]: p

Available ports:
f0/1
Select port to assign IP to: f0/1
Enter IP address: 192.168.10.10
IP set to: 192.168.10.10
Enter subnet mask or prefix (e.g., /24 or 255.255.255.0): 255.211.255.0
Invalid subnet mask
```

Figure 19- Failed Subnet Configuration

```
Select option: 3

Devices on the network:
PC1 : (off)
PC2 : (off)
Which device are you powering on/off: PC1
PC1 is now ON
```

Figure 17- Change Device State

```
except KeyboardInterrupt:
    print("\nOperation cancelled. Returning to menu.")
except Exception as e:
    print("Unexpected error:", e)
```

Figure 20- Error Handling

```
Existing VLANs on this switch:
VLAN 1 -> Ports: f0/1, f0/2, f0/3, f0/4, f0/5, f0/6, f0/7, f0/8, f0/9, f0/

VLAN Menu:
1. Add VLAN
2. Assign ports to VLAN
3. Delete VLAN
X. Exit VLAN Menu
Select an option: 1
Enter VLAN ID (number): 10
Enter VLAN name (optional):
VLAN 10 (VLAN10) added to S1

VLAN Menu:
1. Add VLAN
2. Assign ports to VLAN
3. Delete VLAN
X. Exit VLAN Menu
Select an option: 2
Enter VLAN ID to assign ports to: 10
Available ports:
f0/1, f0/2, f0/3, f0/4, f0/5, f0/6, f0/7, f0/8, f0/9, f0/10, f0/11, f0/12,
Enter ports to assign (comma-separated): f0/1, f0/2, f0/3, f0/4
```

Figure 21- VLAN Configuration

```
Select option: 1
Enter the host (Default: localhost)
Enter the user (Default: root)
Enter the password (Can be blank)
Enter the port number (Default: 3306)
{'host': 'localhost', 'user': 'root', 'password': '', 'database': None, 'port': 3306}
Successfully connected to the database

Database
1. Connect to SQL
2. Create Database/Table
3. Upload Data
4. Show Table
5. Load Network
B. Back

Select option: 2
Please enter a database name: NetworkSim5
Database and Tables already exist
```

Figure 16- Connection to MySQL Database

Appendix B – Sample Code Snippets

Layer 3 Switch Device Object Class

```
class L3Switch(Device):
    canHaveIP = True
    canHaveGateway = False
    canHaveVLAN = True
    models = {
        "3650": {"fastethernet": 24, "gigabitethernet": 4},
        "9300": {"fastethernet": 48, "gigabitethernet": 8},
    }

    def __init__(self, name, model="3650"):
        if model not in self.models:
            raise ValueError(f"Invalid L3Switch model '{model}'")
        ports = (
            Device.generatePorts("f0/", self.models[model]["fastethernet"]) +
            Device.generatePorts("g0/", self.models[model]["gigabitethernet"])
        )
        super().__init__(name, "L3Switch", ports, state="off", model=model)

        # Default all ports to VLAN 1
        for port in self.ports:
            self.ports[port]["vlan"] = 1

        self.vlans = {1: {"name": "default", "ports": list(self.ports.keys()), "ip": None}}

        # --- HSRP support ---
        # Structure: {vlan_id: {"virtual_ip": str, "priority": int, "state": "active"/"standby"}}
        self.hsrp_groups = {}
```

```
def configure_hsrp(self, vlan_id, virtual_ip, priority=100):
    """
    Configures HSRP for a VLAN
    - vlan_id: VLAN number
    - virtual_ip: IP that HSRP group will use
    - priority: determines active/standby (higher = active)
    """
    if vlan_id not in self.vlans:
        raise ValueError(f"VLAN {vlan_id} does not exist on {self.name}")
    self.hsrp_groups[vlan_id] = {
        "virtual_ip": virtual_ip,
        "priority": priority,
        "state": "standby"  # default, you can calculate later
    }

@staticmethod
def determine_hsrp_for_vlan(devices, vlan_id):
    participants = []

    # Find all L3 switches participating in this VLAN
    for dev in devices.values():
        if isinstance(dev, L3Switch) and vlan_id in dev.hsrp_groups:
            participants.append(dev)

    if not participants:
        return

    # Ensure all virtual IPs match
    virtual_ips = {sw.hsrp_groups[vlan_id]["virtual_ip"] for sw in participants}
    if len(virtual_ips) > 1:
```

```
            print(f"Warning: HSRP virtual IP mismatch in VLAN {vlan_id}")
            return

        # Elect active (highest priority)
        active = max(participants, key=lambda sw: sw.hsrp_groups[vlan_id]["priority"])

        # Assign states
        for sw in participants:
            sw.hsrp_groups[vlan_id]["state"] = "active" if sw == active else "standby"
```

Factory Class for Device Creation

```
class Factory: #make classes at runtime
    checkType = {"router","l2switch","l3switch","server","pc", "firewall"}

    @staticmethod
    def normalizeType(deviceType):
        deviceType = deviceType.lower()
        matches = [d for d in Factory.checkType if d.startswith(deviceType)]
        if len(matches) == 1:
            return matches[0]
        else:
            return None

    @staticmethod
    def getAvailableModels(deviceType):
        deviceType = Factory.normalizeType(deviceType)
        if deviceType == "router":
            return list(Router.models.keys())
        elif deviceType == "l2switch":
            return list(L2Switch.models.keys())
        elif deviceType == "l3switch":
```

```
            return list(L3Switch.models.keys())
        else:
            return None

    @staticmethod
    def buildDevice(deviceType, name, model=None):
        deviceType = Factory.normalizeType(deviceType)
        if deviceType == "router":
            return Router(name, model=model if model else "4331")
        if deviceType == "l2switch":
            return L2Switch(name, model=model if model else "2960")
        if deviceType == "l3switch":
            return L3Switch(name, model=model if model else "3650")
        if deviceType == "server":
            return Server(name)
        if deviceType == "pc":
            return PC(name)
        if deviceType == "firewall":
            return Firewall(name)
        else:
            print("Invalid option")
            return None

    def validType(deviceType):
        return Factory.normalizeType(deviceType) is not None
```

Visualizer Hierarchy and Seed Choice

```
plt.figure(figsize=(14, 9))
    for device in devices.values():
        if device.deviceType == "Router":
            G.nodes[device.name]["layer"] = 0
        elif device.deviceType == "Firewall":
            G.nodes[device.name]["layer"] = 1
        elif device.deviceType == "L3Switch":
            G.nodes[device.name]["layer"] = 2
        elif device.deviceType == "L2Switch":
            G.nodes[device.name]["layer"] = 3
        else:
            G.nodes[device.name]["layer"] = 4

    pos = nx.spring_layout(G, k=2, iterations=100, seed=77)
    for node in pos:
        layer = G.nodes[node]["layer"]
        pos[node] = (pos[node][0], -layer)

    labels = nx.get_node_attributes(G, 'label')

    nx.draw_networkx_nodes(G, pos, node_size=4000, node_color=colourMap)
    nx.draw_networkx_labels(G, pos, labels, font_size=7)
```

Ping Function

```
def ping(devices):
    if len(devices) < 2:
        print("Not enough devices on the network")
        return

    print("\nDevices:", ", ".join(devices.keys()))

    device1, p1 = select_device_port(devices, "ping from")
    if not device1:
        return

    device2, p2 = select_device_port(devices, "ping to")
    if not device2:
        return

    src_ip = p1.get("ip")
    dst_ip = p2.get("ip")

    if not src_ip or not dst_ip:
        print("Both devices must have IP addresses")
        return

    print("\nChecking route...")
```

```
visited = set()
queue = deque([(device1, [device1.name])])

while queue:
    current, path = queue.popleft()

    if current.name in visited:
        continue
    visited.add(current.name)

    iface = can_reach_ip(current, dst_ip)
    if iface:
        final_path = path + [f"{current.name}({iface['ip']})"]

        print("\nRoute taken:")
        print(" → ".join(final_path))

        print(f"\nPinging {dst_ip} with 32 bytes of data:")
        for _ in range(4):
            time.sleep(1)
            print(f"Reply from {dst_ip}: bytes=32 time=1ms")

        print(f"Ping successful! {device1.name} can reach {device2.name}")
        return
```

```
        for neighbor in get_neighbors(current, devices):
            if neighbor.name not in visited:
                queue.append((neighbor, path + [neighbor.name]))

    print(f"Ping failed: no route from {device1.name} to {device2.name}")
```

Show Database Tables Function

```
def showTable():
    if dbConfig["database"] is None:
        print("No database selected")
        return

    print("\nDEVICES TABLE:")
    deviceRows = getDevices()
    if deviceRows:
        print(tabulate(deviceRows, headers="keys", tablefmt="grid"))
    else:
        print("No device data found")


    print("\nPORTS TABLE:")
    portRows = getPorts()
    if portRows:
        print(tabulate(portRows, headers="keys", tablefmt="grid"))
    else:
        print("No port data found")

    print("\nCONNECTIONS TABLE:")
```

```
connectionRows = getConnectionsInSQL()
if connectionRows:
    print(tabulate(connectionRows, headers="keys", tablefmt="grid"))
else:
    print("No connection data found")

print("\nETHERCHANNEL TABLE:")
etherRows = getEtherchannels()
if etherRows:
    print(tabulate(etherRows, headers="keys", tablefmt="grid"))
else:
    print("No etherchannel data found")

print("\nVLAN TABLE:")
vlanRows = getVlans()
if vlanRows:
    print(tabulate(vlanRows, headers="keys", tablefmt="grid"))
else:
    print("No VLAN data found")

print("\nHSRP TABLE:")
hsrpRows = getHSRP()
if hsrpRows:
    print(tabulate(hsrpRows, headers="keys", tablefmt="grid"))
else:
    print("No HSRP data found")
```

Establishing Connection to Database

```
def connectToDatabase():
    try:
        dbConfig["host"] = getInput("Enter the host (Default: localhost)") or "localhost"
        dbConfig["user"] = getInput("Enter the user (Default: root)") or "root"
        dbConfig["password"] = getInput("Enter the password (Can be blank)") or ""
        dbConfig["port"] = int(getInput("Enter the port number (Default: 3306)") or 3306)

        print(dbConfig)

        try:
            conn = mysql.connector.connect(
                host=dbConfig["host"],
                user=dbConfig["user"],
                password=dbConfig["password"],
                port=dbConfig["port"]
            )
            conn.close()
        except mysql.connector.Error as e:
            print(f"Error occured while connecting: {e}")
            return False
    except KeyboardInterrupt:
        print("\nConnection attempt cancelled by user")
        return False
    print("Successfully connected to the database")
    return True

def createDatabase():
    dbName = getInput("Please enter a database name: ")
    if dbName is None:
        return
```

```
    dbConfig["database"] = dbName

    conn = mysql.connector.connect(
            host=dbConfig["host"],
            user=dbConfig["user"],
            password=dbConfig["password"],
            port=dbConfig["port"]
    )
    cursor = conn.cursor()

    cursor.execute(f"SHOW DATABASES LIKE '{dbConfig['database']}'")
    dbExists = cursor.fetchone() is not None

    if not dbExists:
        cursor.execute(f"CREATE DATABASE {dbConfig['database']}")
    cursor.close()
    conn.close()

    return not dbExists

def connect():
    if any(v is None for v in dbConfig.values()):
        raise ValueError("Database info not set. Run connectToDatabase() first.")
    return mysql.connector.connect(
        host=dbConfig["host"],
        user=dbConfig["user"],
        password=dbConfig["password"],
        database=dbConfig["database"],
        port=dbConfig["port"] )
```

Preparing Database Table

```
def prepareDatabase():
    if dbConfig["database"] is None:
        return

    conn = connect()
    cursor = conn.cursor()

    # Check/create devices table
    cursor.execute("SHOW TABLES LIKE 'devices'")
    tableExists = cursor.fetchone() is not None
    if not tableExists:
        cursor.execute("""
            CREATE TABLE devices (
                id INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(100) NOT NULL,
                deviceType VARCHAR(50) NOT NULL,
                state VARCHAR(10),
                totalPorts INT,
                usedPorts INT,
                unusedPorts INT,
                model VARCHAR(10)
            )
        """)
```

IP Address Checker

```
def ipChecker(self): #check if ip address is correct format
    ipAddress = self.ipAddress.split(".")

    if len(ipAddress) != 4:
        return False
    else:
        for ipAdd in ipAddress:
            if not ipAdd.isdigit():
                return False

            if len(ipAdd) > 1 and ipAdd.startswith("0"): #ensure ip address doesnt
havce following 0s
                return False

            num = int(ipAdd)
            if num < 0 or num > 255:
                return False

    return True
```

Subnet Mask Checker

```
def subnetChecker(self):
    binary = self.ipToBinary()
    bits = binary.replace(".", "")

    if "01" not in bits:
        return True
    else:
        return False
```

Appendix C – REST API Examples

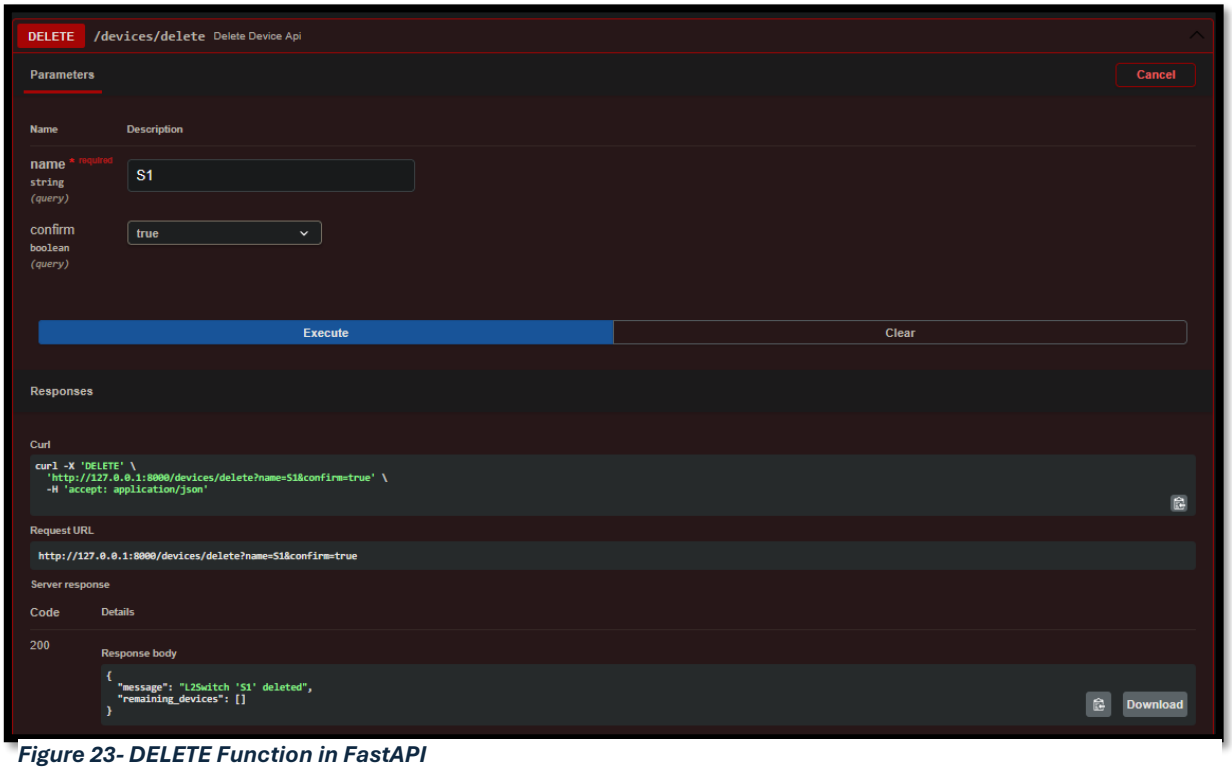


Figure 23- DELETE Function in FastAPI

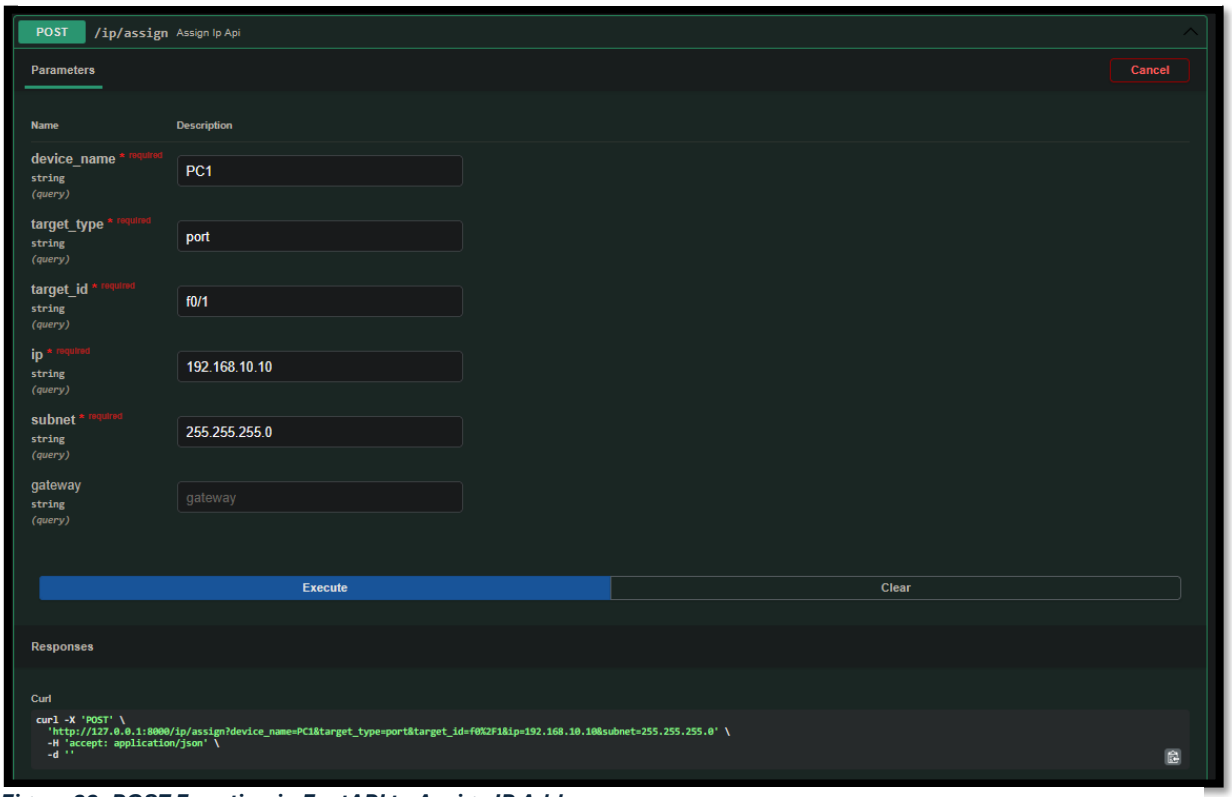


Figure 22- POST Function in FastAPI to Assign IP Address